BÀI TOÁN THỎA MÃN RÀNG BUỘC (CSP)

1. Mục tiêu

- Biểu diễn bài toán Constraint Satisfaction Problem (CSP) thông qua tập biến, miền giá trị và các ràng buộc.
- Cài đặt và kiểm thử Backtracking Search cho bài toán tô màu đồ thị; khảo sát chọn biến MRV, chọn giá trị và Forward Checking.
- Mô hình hóa bài toán n quân hậu bằng số cặp quân hậu xung đột; cài đặt các biến thể Hill Climbing, Random Restart và Simulated Annealing.
- Đánh giá lời giải theo tính hợp lệ, số xung đột, số nút/lần đánh giá, thời gian chạy, tỷ lệ thành công và khả năng mở rộng.
- Tóm tắt kiến thức cần nhớ

Phương pháp	Nguyên tắc	Lưu ý
Backtracking	Gán biến từng bước, quay lui khi vi phạm ràng buộc	Đầy đủ với CSP hữu hạn khi duyệt hết khả năng.
MRV / Forward Checking	Chọn biến ít giá trị hợp lệ; loại sớm giá trị xung đột	Giảm nhánh tìm kiếm, có chi phí quản lý miền.
Min-Conflicts	Sửa phép gán đầy đủ theo số ràng buộc vi phạm	Tìm kiếm cục bộ, không bảo đảm chứng minh vô nghiệm.
Hill Climbing	Chỉ nhận nước đi cải thiện	Có thể kẹt tại cực trị cục bộ.
Random Restart	Tìm kiếm lại từ trạng thái khởi tạo khác	Tăng cơ hội tìm nghiệm, tăng tổng chi phí.
Simulated Annealing	Đôi khi chấp nhận nước đi tệ hơn theo nhiệt độ	Phụ thuộc lịch làm nguội và điều kiện dừng.

2. Dữ liệu

Nhóm	Tập tin	Nội dung
Tô màu đồ thị	CSP_graph_coloring_example.ipynb	Sinh đồ thị phẳng bằng Delaunay; CSP với 3/4 màu; kiểm tra complete/consistent; Backtracking; bài tập MRV, Forward Checking, Min-Conflicts.
N quân hậu	n_queens.ipynb	Biểu diễn bàn cờ, random_board(), conflicts(), show_board(); 6 nhiệm vụ thuật toán và đánh giá, cùng bài nâng cao.
Hướng dẫn	README.md	Giới thiệu hai notebook và mối liên hệ giữa CSP với tìm kiếm cục bộ.

3. Bài tập thực hành

Bài 1. Biểu diễn CSP và sinh đồ thị cần tô màu

- Mở CSP_graph_coloring_example.ipynb, chạy các cell tạo n = 10 điểm ngẫu nhiên và phép tam giác hóa Delaunay. Quan sát hình đồ thị và danh sách neighbors.
- Xác định X (các đỉnh), D (tập màu), C (hai đỉnh kề nhau phải khác màu); giải thích vì sao chỉ tạo ràng buộc cho cạnh i < j.
- Khảo sát domain gồm 4 màu và 3 màu; giải thích sự khác nhau giữa một phép gán đầy đủ và một phép gán hợp lệ.

```
Mô hình: X = {0, 1, …, n-1}; Dᵢ = {red, blue, green, orange}; Cᵢⱼ: color(i) ≠ color(j) với (i,j) là cạnh.
```

Bài 2. Kiểm tra tính đầy đủ và nhất quán

- Chạy complete(assignment, csp) và consistent(assignment, csp) với phép gán rỗng, một phần, đầy đủ và phép gán có hai đỉnh kề cùng màu.
- Viết ít nhất 3 ca kiểm thử và ghi rõ kết quả mong đợi; nhận xét vì sao complete() không đồng nghĩa consistent().

© DNTAI

- Xác định dạng lưu trữ csp gồm variables, domains và constraints; phân biệt phép gán từng phần và nghiệm CSP.

Bài 3. Backtracking Search và thứ tự chọn biến

- Chạy backtrack_search(csp), theo dõi biến COUNT (số nút được kiểm tra), thời gian chạy và kết quả tô màu. Trực quan hóa bằng đoạn mã vẽ trong notebook.
- Phân tích select_unassigned_var(): phiên bản ban đầu chọn biến chưa gán đầu tiên; cài đặt Minimum Remaining Values (MRV) trên miền còn hợp lệ.
- Thử bổ sung một chiến lược thứ tự giá trị (ví dụ Least Constraining Value – LCV); so sánh kết quả với phiên bản gốc trên cùng đồ thị.

Bài 4. Forward Checking và Min-Conflicts

- Bổ sung Forward Checking: sau khi gán màu cho một đỉnh, loại màu xung đột khỏi miền các đỉnh láng giềng chưa gán; hoàn tác miền khi quay lui.
- Cài đặt tìm kiếm cục bộ Min-Conflicts theo yêu cầu bài Assignment: khởi tạo một phép gán đầy đủ, chọn biến đang vi phạm rồi đổi sang giá trị làm giảm số ràng buộc vi phạm.
- Với n khác nhau và miền 3/4 màu, tạo nhiều đồ thị ngẫu nhiên; so sánh tỷ lệ tìm nghiệm, COUNT hoặc số bước, thời gian chạy. Lưu ý bài toán 3 màu có thể vô nghiệm.

Bài 5. Biểu diễn bài toán n quân hậu

- Mở n_queens.ipynb; khảo sát random_board(n), conflicts(board), show_board(board) và bàn cờ mẫu [1, 3, 0, 2].
- Mỗi cột có đúng một quân hậu; board[i] là chỉ số hàng của quân ở cột i. Viết rõ hàm mục tiêu h(board) = số cặp quân hậu cùng hàng hoặc đường chéo.
- Tạo các bàn cờ n = 4 và n = 8; hiển thị bàn cờ cùng h(board), kiểm tra h(board) = 0 khi và chỉ khi không có cặp quân hậu tấn công nhau.

Bài 6. Các biến thể Hill Climbing

- Task 1: cài đặt Steepest-ascent Hill Climbing, đánh giá toàn bộ các nước đi lân cận, chọn nước cải thiện tốt nhất và dừng tại cực trị cục bộ.
- Task 2: cài đặt Inefficient Stochastic Hill Climbing bằng cách chọn ngẫu nhiên một nước đi trong tất cả các nước cải thiện.
- Task 3: cài đặt Efficient Stochastic Hill Climbing: chỉ sinh lần lượt một láng giềng ngẫu nhiên và chấp nhận khi cải thiện; xác định điều kiện dừng rõ ràng để tránh lặp vô hạn.
- Ghi nhận giá trị xung đột cuối, số bước, số lần đánh giá hàm mục tiêu và trường hợp mắc kẹt ở local optimum/plateau.

Bài 7. Random Restart và Simulated Annealing

- Task 4: chạy lại ba biến thể Hill Climbing từ các bàn cờ khởi tạo ngẫu nhiên, tối đa 100 lần khởi động lại; thống kê số lần thành công và chi phí tính toán.
- Task 5: cài đặt Simulated Annealing, thử ít nhất hai lịch làm nguội; nghiệm tệ hơn có thể được chấp nhận theo nhiệt độ và độ chênh hàm mục tiêu.
- Giữ cùng n và điều kiện dừng hợp lý khi so sánh các thuật toán; phân biệt chạy thành công (h = 0) với nghiệm tốt nhất (h nhỏ nhất).

```
Ví dụ: p = exp(−Δh/T) khi Δh > 0; T giảm dần theo số vòng lặp.
```

Bài 8. Đánh giá tổng hợp và bài nâng cao

- Task 6: chạy tối thiểu 100 lượt với các bàn cờ khởi tạo khác nhau tại n = 4 và n = 8; so sánh thời gian, h cuối, tỷ lệ thành công và trình bày kết quả bằng bảng/biểu đồ.
- Vẽ đồ thị hội tụ h theo số vòng lặp cho một lượt chạy đại diện của mỗi thuật toán tại n = 8; nhận xét tốc độ cải thiện và plateau.

© DNTAI

- Vẽ biểu đồ log-log thời gian chạy theo n ∈ {4, 8, 12, 16, 20} cho ít nhất hai thuật toán; thảo luận độ phức tạp thực nghiệm và sự phụ thuộc vào cài đặt.
- Nâng cao (tùy chọn): khảo sát toán tử di chuyển cục bộ khác trong phần Advanced task và so sánh với toán tử ban đầu.

4. Hướng dẫn thực hiện

4.1 Quy trình chạy notebook

1. Giải nén lab04 - csp.zip, mở hai notebook bằng JupyterLab/Jupyter Notebook hoặc Google Colab.

2. Cài đặt những gói sử dụng trong notebook nếu môi trường chưa có (numpy, scipy, matplotlib); thực thi từ đầu theo thứ tự cell.

3. Với notebook tô màu đồ thị, giữ cố định hạt giống ngẫu nhiên khi so sánh từng thuật toán trên cùng dữ liệu; sau đó thay đổi nhiều seed để đánh giá độ ổn định.

4. Với n_queens.ipynb, điền mã và mô tả vào các cell "Code and description go here"; không coi các Task chưa được cài đặt là thuật toán sẵn có.

5. Dùng %time hoặc time.perf_counter() để đo thời gian; ghi các điều kiện thí nghiệm, số lần chạy và tiêu chí dừng; lưu hình trực quan cùng đầu ra notebook.

4.2 Bảng ghi kết quả thí nghiệm

Bài toán	Kích thước	Thuật toán	Số màu / lượt	Thành công (%)	Xung đột / COUNT	Thời gian (s)
Tô màu	n = 10	Backtracking	4			
Tô màu	n = 10	MRV + Backtracking	4			
Tô màu	n = 10	Forward Checking	3 / 4			
Tô màu	n thay đổi	Min-Conflicts	3 / 4			
n-Queens	n = 4	Steepest HC	≥100			
n-Queens	n = 8	Stochastic HC	≥100			
n-Queens	n = 8	Random Restart	≥100			
n-Queens	n = 8	Simulated Annealing	≥100			

Ghi rõ số seed/lượt thử, tiêu chí dừng và cách tính trung bình; COUNT ở bài tô màu không đồng nhất với số lần đánh giá hàm mục tiêu ở n-Queens.

4.3 Câu hỏi thảo luận

1. Khác nhau giữa complete() và consistent()? Một phép gán đầy đủ có luôn là nghiệm CSP không?

2. MRV và Forward Checking tác động thế nào đến cây Backtracking? Khi nào kiểm tra miền làm tăng chi phí?

3. Vì sao tô màu bằng 3 màu có thể thất bại dù dùng thuật toán đúng? Làm sao phân biệt hết giới hạn tính toán và chứng minh vô nghiệm?

4. Hill Climbing gặp local optimum hoặc plateau ra sao? Random Restart và Simulated Annealing xử lý theo cách nào?

5. So sánh Steepest, Inefficient Stochastic và Efficient Stochastic về số lần đánh giá các láng giềng.

6. Vì sao cần nhiều lần chạy và cố định điều kiện thí nghiệm khi đánh giá thuật toán ngẫu nhiên?

© DNTAI

5. Yêu cầu sản phẩm nộp

- Hai notebook CSP_graph_coloring_example.ipynb và n_queens.ipynb đã hoàn thiện các phần bài tập, có Markdown giải thích thuật toán và kết quả chạy.
- Bảng tổng hợp thí nghiệm cho tô màu đồ thị (3/4 màu, nhiều kích thước) và n-Queens (n = 4/8, ít nhất 100 lượt mỗi thiết lập so sánh).
- Ít nhất hai hình kết quả gồm đồ thị tô màu/bàn cờ n quân hậu và hai biểu đồ phân tích hội tụ hoặc thời gian chạy.
- Nhận xét ngắn về tính hợp lệ nghiệm, hiệu quả thuật toán, các trường hợp thất bại và hạn chế của đánh giá thực nghiệm.
- Ghi thông tin sinh viên/nhóm, môi trường chạy, các thư viện sử dụng; notebook phải chạy lại được từ đầu mà không phụ thuộc đường dẫn cá nhân.

---o0o---

(Hết)

© DNTAI